
Set 3

OPENMP and GPUs 2024-2025

Software & Programming of HPC Systems
(CEID_NE5407)

Στοιχεία Φοιτητών

Υπεύθυνος Καθηγητής:

Παναγιώτης Χατζηδούκας, Ευάγγελος Δερματάς, Ευστράτιος Γαλλόπουλος

Μέλος 1

Ακαδημαϊκό Έτος : 9^ο Εξάμηνο 2023-2024

Ονοματεπώνυμο: Ορέστης Αντώνιος Μακρής **A-M** 1084516

Μέλος 2

Ακαδημαϊκό Έτος : 9^ο Εξάμηνο 2023-2024

Ονοματεπώνυμο: Γρηγόρης Δελημπαλταδάκης **A-M** 1084647

Πανεπιστήμιο Πατρών Τμήμα Μηχανικών Η/Υ και Πληροφορικής

Πάτρα 2025

Contents

Question 1: OpenMP and Complex Matrix Multiplication.....	3
Επισκόπηση Δομής για το Ερώτημα 1	3
Πειράματα Βελτιστοποίησης.....	3
Εξήγηση Πειραμάτων.....	3
Περιγραφή Βέλτιστης Υλοποίησης	4
Εξήγηση Κώδικα.....	4
Τεχνικές Βελτιστοποίησης	5
Πειραματική Αξιολόγηση.....	6
Πίνακας Αποτελεσμάτων	6
Διάγραμμα Αποτελεσμάτων	6

Question 1: OpenMP and Complex Matrix Multiplication

Επισκόπηση Δομής για το Ερώτημα 1

Όλα τα αρχεία που υλοποιήθηκαν για την άσκηση βρίσκονται στον φάκελο **Code**.

Code

- complex_mat_mul.c
- helper_functions.h
- Makefile
- strength_reduct_compl_mat_mul.c
- tiled_complex_mat_mul.c

Σε αυτό τον φάκελο έχουν τοποθετηθεί διαφορετικές υλοποιήσεις για τον πολλαπλασιασμό μιγαδικών μητρώων, δοκιμάζοντας διαφορετικές τεχνικές βελτιστοποίησης. Η τελική και βέλτιστη υλοποίηση βρίσκεται στο αρχείο **tiled_complex_mat_mul.c**.

Για την εκτέλεση οποιασδήποτε υλοποίησης μπορείτε να μεταβείτε στον φάκελο Code και να τρέξετε:

```
$ make <source code file name>
```

```
$ ./<executable> <matrix dimension>
```

Για παράδειγμα:

```
hpcgrp00@krylov100:~/HPC-2024/Set_3/Code$ ./tiled_complex_mat_mul 1024
GPU Execution Time (Tiled): 0.490666 seconds
CPU Execution time: 25.589410 seconds
Verifying results...
Complex Matrix Multiplication is correct!
```

Πειράματα Βελτιστοποίησης

Ο παρακάτω πίνακας παρουσιάζει τα πειράματα που πραγματοποιήθηκαν, μαζί με τους αντίστοιχους χρόνους εκτέλεσης σε κάθε περίπτωση.

Σημειώνεται ότι καταγράφεται ο χρόνος εκτέλεσης στην GPU και ο χρόνος μεταφοράς δεδομένων μεταξύ GPU και CPU.

Ο πολλαπλασιασμός πραγματοποιείται για μητρώα διάστασης $N = 2048$.

<i>Experiment</i>	<i>GPU Execution Time (sec)</i>
Complex matrix multiplication, using Global Memory .	18.92
Complex matrix multiplication, using Strength Reduction	19.95
Tiled complex matrix multiplication.	3.62

Εξήγηση Πειραμάτων

- Αρχικά, υλοποιήσαμε το ερώτημα χωρίς tiling. Δηλαδή, τα τελικά μητρώα E και F υπολογίζονται, με τα μητρώα A, B, C, D να προσπελούνται από την global memory. Η συγκεκριμένη υλοποίηση αποτελεί ένα baseline για τα υπόλοιπα πειράματα.
- Στην συνέχεια, δοκιμάσαμε να εφαρμόσουμε strength reduction, υπολογίζοντας τα ενδιάμεσα μητρώα P, Q, R , όπως και στο Ερώτημα 2 του Set 2. Αυτή η τεχνική όμως οδήγησε σε αύξηση του χρόνου εκτέλεσης.

3. Στην συνέχεια δοκιμάσαμε να χωρίσουμε τα μητρώα σε tiles, ώστε κάθε thread block (team) να υπολογίζει ένα μέρος του τελικού μητρώου.
Αυτή η βελτιστοποίηση μείωσε τον χρόνο εκτέλεσης κατά **80%**.

Είναι προφανές ότι παρόλο που η OpenMP δεν υποστηρίζει έλεγχο της GPU shared memory, το tiling αυξάνει την απόδοση, βελτιώνοντας το cache locality.

Περιγραφή Βέλτιστης Υλοποίησης

Η υλοποίηση του ερωτήματος της άσκησης βρίσκεται στο αρχείο **Code/ tiled_complex_mat_mul.c**.

Εξήγηση Κώδικα

Είσοδοι και Αρχικοποίηση

Ο σκοπός της άσκησης είναι ο υπολογισμός των μητρώων $E = AC - BD$ και $F = AD + BC$.

Αρχικοποιούμε 4 μητρώα **A, B, C, D**, μεγέθους NxN, με τυχαίες ακέραιες τιμές μεταξύ του 0 και 99. Παράλληλα κάνουμε allocate τα μητρώα **E_gpu, F_gpu**, στα οποία θα αποθηκευτούν τα αποτελέσματα που υπολογίζονται στην GPU και τα μητρώα **E_cpu, F_cpu**, για τον υπολογισμό στην CPU.

OpenMP Target Offloading

Ο υπολογισμός αναθέτεται στην GPU, χρησιμοποιώντας το **target** directive. Συγκεκριμένα κάνουμε map τα μητρώα A, B, C, D στην GPU και αντιγράφουμε τους τελικούς πίνακες E_gpu και F_gpu πίσω στον host μετά τον υπολογισμό τους.

```
#pragma omp target data map(to: A[0:N*N], B[0:N*N], C[0:N*N], D[0:N*N]) \
                        map(from: E_gpu[0:N*N], F_gpu[0:N*N])
```

Loading Tiles

Μέσα σε ένα **teams distributed parallel for loop** κάνουμε iterate πάνω στο μητρώο, με **TILE_SIZE*TILE_SIZE** βήματα. Σε κάθε βήμα αρχικοποιούμε προσωρινά μητρώα αποτελέσματος μεγέθους TILE_SIZE για τα E, F (**localE, localF**) και τα μητρώα **tileA, tileB, tileC** και **tileD** για την προσπέλασή των απαραίτητων tiles από τα αρχικά A, B, C, D. Μετά την αρχικοποίηση, φορτώνουμε τα απαραίτητα tiles από τα αρχικά μητρώα εισόδου.

Π.χ. για τα tiles των A και B

```
for (int k = 0; k < N; k += TILE_SIZE) {
    // Load tile for A and B
    for (int ti = 0; ti < TILE_SIZE; ti++) {
        for (int tk = 0; tk < TILE_SIZE; tk++) {
            int global_row = i + ti, global_col = k + tk;
            if (global_row < N && global_col < N) {
                tileA[ti][tk] = A[global_row * N + global_col];
                tileB[ti][tk] = B[global_row * N + global_col];
            } else {
                tileA[ti][tk] = 0;
                tileB[ti][tk] = 0;
            }
        }
    }
}
```

Tile Computation

Έχοντας φορτώσει τα tiles, υπολογίζονται τα απαραίτητα εσωτερικά γινόμενα για το τρέχον tile.

```
// Compute partial tile results
for (int ti = 0; ti < TILE_SIZE; ti++) {
    for (int tj = 0; tj < TILE_SIZE; tj++) {
        int sumAC = 0, sumBD = 0;
        int sumAD = 0, sumBC = 0;
        for (int tk = 0; tk < TILE_SIZE; tk++) {
            sumAC += tileA[ti][tk] * tileC[tk][tj];
            sumBD += tileB[ti][tk] * tileD[tk][tj];
            sumAD += tileA[ti][tk] * tileD[tk][tj];
            sumBC += tileB[ti][tk] * tileC[tk][tj];
        }
        local_E[ti][tj] += sumAC - sumBD;
        local_F[ti][tj] += sumAD + sumBC;
    }
}
```

Writing Back Results

Τέλος τα υπολογισμένα tiles γράφονται πίσω στα global μητρώα E_gpu και F_gpu.

```
// Write back computed tile to global memory
for (int ti = 0; ti < TILE_SIZE; ti++) {
    for (int tj = 0; tj < TILE_SIZE; tj++) {
        int global_row = i + ti, global_col = j + tj;
        if (global_row < N && global_col < N) {
            E_gpu[global_row * N + global_col] = local_E[ti][tj];
            F_gpu[global_row * N + global_col] = local_F[ti][tj];
        }
    }
}
```

Επαλήθευση

Τελικά, μέσω της βοηθητική συνάρτησης **verify_complex_mat_mul**, ο πολλαπλασιασμός εκτελείται σειριακά και στην CPU και τα αποτελέσματα συγκρίνονται για επαλήθευση.

```
verify_complex_mat_mul(A, B, C, D, E_cpu, E_gpu, F_cpu, F_gpu, N);
```

Τεχνικές Βελτιστοποίησης

Παραλληλοποίηση με teams distribute parallel for

Το directive **#pragma omp target teams distribute parallel for collapse(2)**, συμβάλει στην αποδοτική παραλληλοποίηση του loop, αξιοποιώντας τα GPU threads.

Συγκεκριμένα το **teams distribute**, διαμοιράζει τον φόρτο εργασίας σε πολλαπλά GPU blocks (teams), το **parallel for**, παραλληλοποιεί τον εσωτερικό loop αξιοποιώντας τα GPU threads, ενώ το **collapse(2)**, συνδυάζει τα δύο εξωτερικά loops σε ένα, βελτιώνοντας το παραλληλισμό.

Tiling

Η χρήση tiling συμβάλει στην αποσυμφόρηση της κίνηση δεδομένων από την global memory. Τα πειραματικά αποτελέσματα δείχνουν πως το OpenMp αξιοποιεί την shared memory της GPU, επιτρέποντας μας να φορτώσουμε tiles των μητρώων εισόδου και να χρησιμοποιήσουμε για πολλούς υπολογισμούς ταχύτερα. Συμπληρωματικά, το tiling διασφαλίζει coalesced memory access, όπου τα threads έχουν πρόσβαση σε συνεχείς θέσεις μνήμης, μεγιστοποιώντας το memory throughput.

Περιορισμός Μεταφοράς Δεδομένων μεταξύ Host-Device

Η χρήση **map(to:)** και **map(from:)**, διασφαλίζει την μεταφορά δεδομένων από και προς την GPU μόνο όταν αυτό είναι απαραίτητο. Τα ενδιάμεσα αποτελέσματα των **E_gpu** και **F_gpu** παραμένουν στην GPU μέχρι να ολοκληρωθούν όλοι οι υπολογισμοί.

Πειραματική Αξιολόγηση

Παρακάτω παραθέτουμε έναν πίνακα με πειραματικές μετρήσεις του χρόνου εκτέλεσης του πολλαπλασιασμού μιγαδικών πινάκων για διαφορετικές διαστάσεις πινάκων στην GPU και CPU.

Πίνακας Αποτελεσμάτων

<i>N</i>	<i>GPU Execution Time (seconds)</i>	<i>Serial CPU Execution Time (seconds)</i>
512	0.066	2.779
1024	0.472	26.236
2048	3.618	458.201
4096	29.675	4380.671

Διάγραμμα Αποτελεσμάτων

